

Survival Model Evaluation: synthetic

Fitted with the survival-analysis-pipeline, on synthetic data with known ground truth

Source data	<code>data/strategies.csv</code> , drawn at seed 7
Rows	5,000 (93.2% with the ending observed, 6.8% censored)
Start dates	2021-07-01 to 2026-06-01
Evaluation	5 expanding-window temporal folds
Source of figures	<code>reports/synthetic/metrics.json</code> , regenerated by the command in section 9

1. Summary

This report evaluates two survival models fitted to synthetic data drawn at seed 7. It holds 5,000 strategies, each observed from its start date. 93.2% have observed endings and 6.8% are censored, still running when observation stopped. Median observed duration, censored strategies included, is 91 days. The models predict, from what was on file at the start date, how long each strategy survives.

The like-for-like comparison is the fold mean. Each of the 5 temporal folds trains both models on one stretch of history and tests them on the next, and the 5 scores average. On concordance, the share of pairs a ranking orders correctly where 0.500 is a coin flip, XGBoost AFT scores 0.781 and the Cox proportional hazards baseline 0.780. The two models effectively tie.

Pooled over 3,000 out-of-time test strategies, the AFT model scores 0.781 (95% bootstrap interval 0.772 to 0.789). Section 5 explains how the two figures differ.

Both models are saved in one bundle, which records the boosted model as recommended for scoring new rows.

Little of the pooled figure is `asset_class` group membership. Ranking strategies by their group's average prediction alone scores 0.538, and comparing only strategies inside the same group scores 0.779, clear of the coin flip. Section 5 gives the decomposition.

An oracle ranking (Addendum A) bounds every model at 0.820. Ranking by validation Sharpe scores 0.410, below the coin flip in every fold.

All results are synthetic. The run validates the pipeline and asserts nothing about live data.

2. Motivation

From the run's notes, written by the analyst.

An automated strategy search produces a queue of candidates that all clear validation, and they stop working at very different rates. How much capital a new strategy should receive, when its first review should be scheduled, and when it should be retired all depend on how long its edge will last. Treating every deployment the same misallocates in both directions. It overfunds strategies that will not survive the quarter and underfunds ones that would have run for a year.

The question this run poses, on a synthetic population built to mimic that setting, is whether discovery-time metadata carries enough signal to separate them. A strategy selected from a hundred thousand candidates carries more selection bias than one selected from a hundred, and a strategy whose walk-forward Sharpe was already sliding during validation had begun decaying before it was deployed. None of that is visible in a single validation statistic, and all of it is already recorded. On a real book, death would be a bookkeeping event, the date a strategy stops clearing the book's retention rule. The lifetimes a model learns there are partly a property of that rule. The generator abstracts this to a lifetime drawn at birth.

3. Data

Durations are measured in days.

6% of strategies are administratively retired independent of performance, and survival time is log-normal in the latents at scale 0.55.

From the run's notes, written by the analyst.

The generator installs detectable structure in the data on purpose, so that recovering it is a test rather than an interpretation. Each of the six feature families shifts log survival time by a fixed amount, microstructure the most fragile and value-carry the most durable. Among the four asset classes, crypto is built to decay fastest, a shift of -0.30 on log time against the fx-majors baseline. It is drawn for roughly 15 percent of rows, so the class effect has to be recovered from a minority slice.

Censoring concentrates among recently discovered strategies, which have had the least time to fail before the observation cutoff, so the final fold's test block is far more censored than the population overall.

Figure 1 shows Kaplan-Meier survival curves by `asset_class`, the coarsest structure in the outcome before any model is fitted.

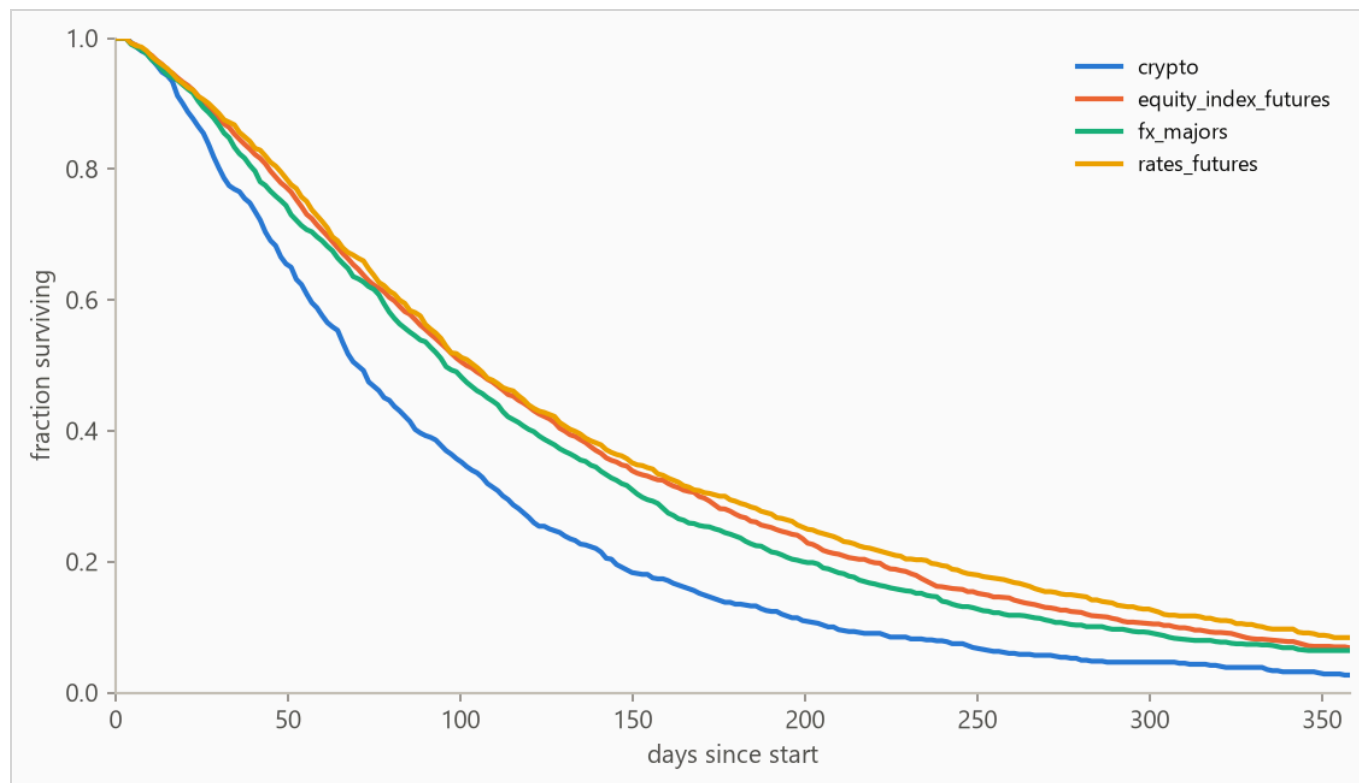


Figure 1. Kaplan-Meier survival curves by `asset_class`: the fraction of each group still running at each age, with censored strategies counted for as long as they were observed. Groups beyond the seven most frequent, where present, are collapsed into (other) for this plot only.

4. Method

4.1 Model class

The pipeline fits two models on every run. The first is a boosted-tree accelerated failure time (AFT) model, XGBoost with its `survival:aft` objective, built for durations with incomplete observations. An observed ending tells the model the exact lifetime, and a censored strategy tells it only "at least this long", so every strategy contributes. It predicts each strategy's median survival time in days, and a log-normal curve around that median, whose width is fitted once and shared by every strategy, gives the probability of surviving any horizon. The second is a Cox proportional hazards baseline, the standard linear survival model, fitted with lifelines' `CoxPHFitter` on the same features. It answers whether the boosted model was necessary. The run's recommended model is whichever scores the higher fold-mean concordance.

The two differ in what they assume. The AFT model assumes every strategy follows the same survival curve run on a faster or slower clock, so features change when things happen, never the curve's shape. The Cox model makes no assumption about that shape at all, reading the curve from the data by counting who was still running at each age. In exchange it assumes each feature multiplies risk by the same factor at every age, which this report does not test. Which set of assumptions suits a dataset cannot be known in advance, which is why both are fitted and scored.

Numeric features pass through, missing values included (the Cox baseline gets train-window median imputation). Text columns are one-hot encoded with the vocabulary refit per training window, so a fold's features reflect only what was on file by its split date.

4.2 Temporal validation and label re-censoring

Evaluation uses 5 expanding-window folds ordered by start date: the earliest 40% of strategies is burn-in, trained on but never tested, and each fold trains on every strategy started before its split date and tests on the next block (sizes in Table 2).

Training labels are re-censored at each split date. A strategy started long before a split may have died after it, and its label contains that future, so every post-split death is rewritten as a censoring at the split. Omitting this raises scores by importing the future, which makes it the most consequential detail in the pipeline. It is a standalone function (`temporal_folds.recensor`) with dedicated tests.

4.3 Selection and calibration

The boosted model's hyperparameters are selected once on the first fold's training window by an inner temporal split, the same past-then-future cut made inside that window. Concordance grades only whether strategies are ordered correctly, and a model can order them well while drawing survival curves far too wide or too narrow. So selection is scored on held-out censored log-likelihood instead, which grades the whole predicted distribution against what was observed.

The predictive scale is the width of the boosted model's log-normal curve, one unitless number shared by every strategy. A larger scale spreads the model's probability over a wider range of lifetimes. Training used 0.6, a setting of the loss that shapes how the medians are fitted. Then, with the medians held fixed, the width that best matches held-out outcomes on the training window's most recent stretch was measured at 0.64 and carried to the model refitted on the full window. The two answer different questions, so they need not agree. Every probability the boosted model reports here uses the measured value.

5. Results

5.1 Discrimination

Table 1. Concordance index by model, pooled over 3,000 test strategies with 95% percentile bootstrap intervals over 500 resamples. Higher is better, and 0.500 is a coin flip. Fold-mean rows score each of the 5 folds separately and average. The interval resamples test rows with the fitted models held fixed, so it measures scoring precision, not stability across history. The fold spread is the guide to that.

METHOD	CONCORDANCE (HARRELL'S C)	95% INTERVAL
Oracle ranking on latent log-time (ceiling)	0.820	not resampled
XGBoost AFT (pooled)	0.781	0.772 to 0.789
XGBoost AFT (fold mean)	0.781	not resampled
Cox proportional hazards (fold mean)	0.780	not resampled
Rank by validation Sharpe	0.410	0.397 to 0.422

Each fold fits its own models. The boosted model predicts a survival time in days, and days mean the same thing in every fold, so its predictions can be pooled into one list. A Cox score is a risk relative to the other strategies in its own fit, so Cox scores from different folds cannot go in one list. The two models are therefore compared on fold means, each fold scored on its own and the 5 scores averaged.

The pooled row answers a different question. It scores all test strategies in one list, which is enough data to attach an uncertainty range, and that range is the interval beside it. It is not the number for comparing models, because its strategies were scored by 5 different fitted models. On this run it matches the fold mean at the printed precision.

Folds 2 and 3 are the boosted model's weakest, at 0.768 and 0.767, too close to separate.

The pooled concordance decomposes by `asset_class`. Ranking strategies by their group's average gives every strategy in a group the same score, so comparisons only ever run between groups, and those come out right 53.8% of the time. The boosted model scores each strategy individually instead, which orders strategies inside a group and can also move one past strategies in other groups. Inside a group it is right 77.9% of the time, averaged over the 4 groups big enough to score (at least 50 strategies and 10 observed endings), each weighted by its number of comparable pairs.

Validation-Sharpe ranking scores 0.410 pooled, 0.354 to 0.427 across folds, never above 0.500. Reversed, it scores 0.590, its arithmetic complement, short of the model's 0.781.

Figure 2 plots the per-fold concordance from Table 2.

Table 2. Per-fold results. Censoring is the share of each fold's test strategies whose ending was not observed.

FOLD	SPLIT DATE	TRAIN N	TEST N	CENSORED	AFT	COX	SHARPE	ORACLE
1	2023-06-22	1,999	600	2%	0.781	0.783	0.425	0.824
2	2024-02-03	2,600	600	2%	0.768	0.769	0.421	0.808
3	2024-09-01	3,199	600	2%	0.767	0.764	0.413	0.805
4	2025-04-03	3,800	600	9%	0.773	0.769	0.427	0.811
5	2025-11-03	4,399	600	39%	0.817	0.815	0.354	0.846

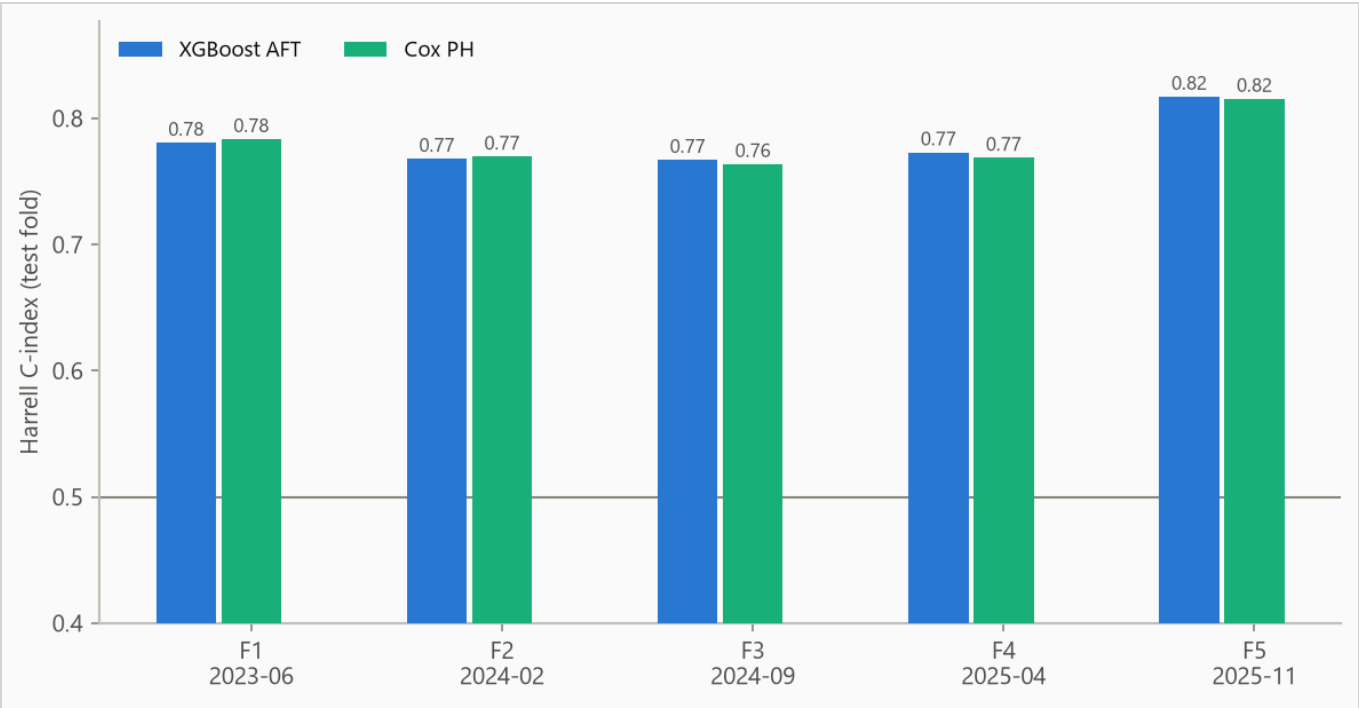


Figure 2. Concordance by fold. The boosted model spans 0.767 to 0.817. Folds differ in censoring mix, so cross-fold comparisons are indicative rather than exact.

5.2 Calibration

The Brier score, the mean squared error of a probability forecast (lower is better), is weighted by the inverse probability of censoring (IPCW) so censored strategies do not bias it. The no-skill reference assigns every strategy one population-wide probability, the Kaplan-Meier marginal, the fraction of the whole population surviving at each age with censored strategies counted while observed.

Table 3. IPCW Brier score by horizon. Lower is better. The weighting works by counting each strategy still observed at a horizon, weighted up by one over the probability that observation lasted that long, so the strategies censoring removed are represented by those it spared.

HORIZON	AFT	COX	NO-SKILL MARGINAL
90 days	0.149	0.151	0.249
180 days	0.132	0.133	0.182
365 days	0.050	0.051	0.056

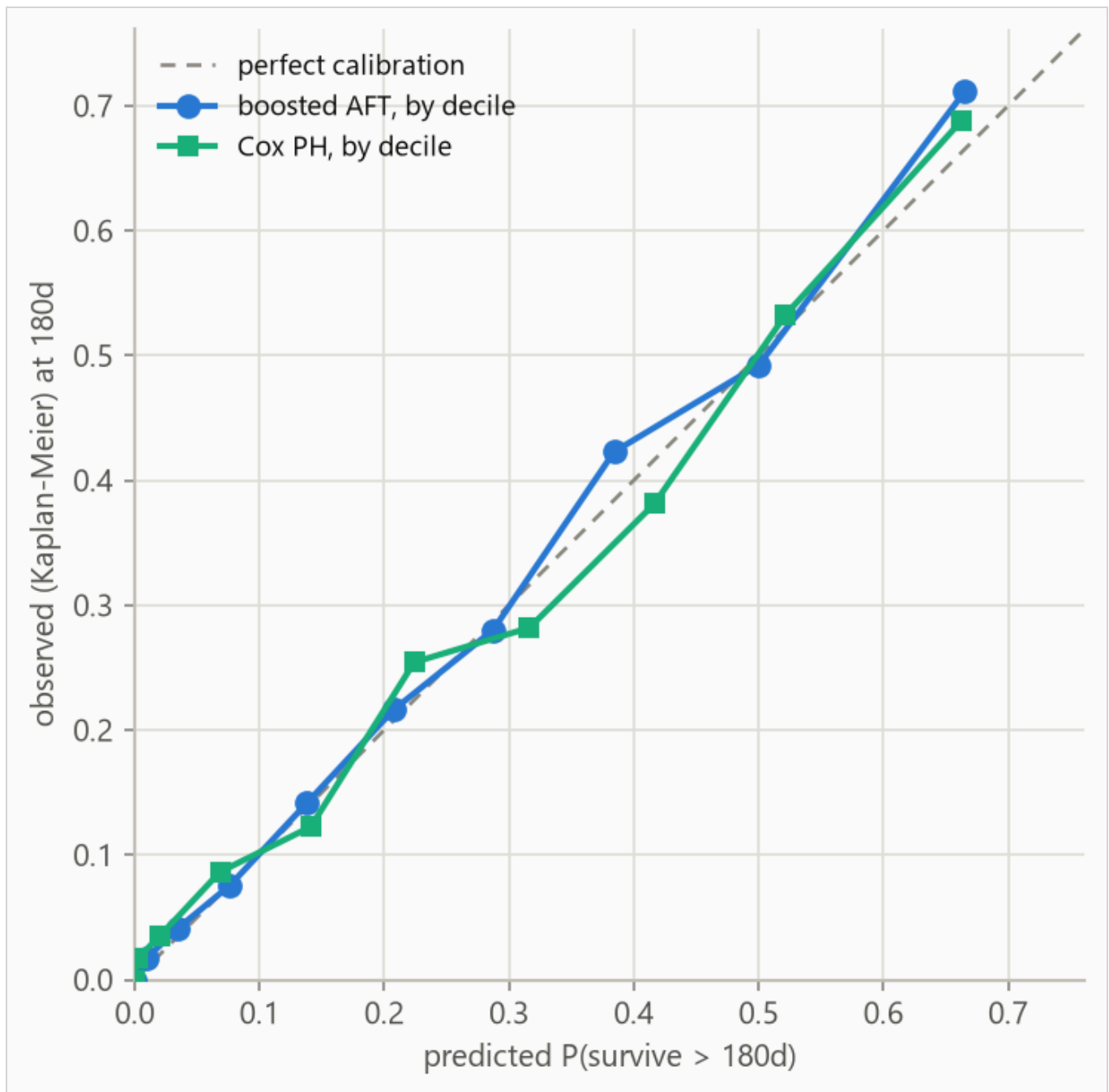


Figure 3. Predicted against observed survival at 180 days for both models, each binned on its own predicted deciles. Observed frequencies are Kaplan-Meier estimates within each bin, so censored strategies contribute correctly. The largest deviation is 0.046 in the boosted model's decile 10 and 0.035 in the Cox baseline's decile 8.

Table 4. Decile calibration at 180 days for the boosted AFT model, the values plotted as its series in Figure 3. Deciles are cut on predicted value, so tied predictions make the counts uneven. The observed value in each is a Kaplan-Meier estimate. When a decile's last observed strategy falls short of the 180-day horizon the estimate carries its last value forward, so in small or heavily censored deciles the observed column carries more uncertainty than its three decimals suggest.

DECILE	N	PREDICTED	OBSERVED (KM)	DEVIATION
1	300	0.001	0.000	-0.001
2	300	0.010	0.017	+0.008
3	300	0.035	0.041	+0.006
4	300	0.076	0.076	-0.001
5	300	0.138	0.142	+0.003
6	300	0.208	0.216	+0.009
7	300	0.287	0.280	-0.007
8	300	0.384	0.423	+0.039
9	300	0.500	0.493	-0.008
10	300	0.665	0.712	+0.046

6. What the models use

6.1 The boosted model

Feature attributions (SHAP values, for SHapley Additive exPlanations) are computed for the boosted model on the log scale of survival time. A +0.3 attribution multiplies predicted survival by about 1.35, and negative values shorten it.

Attributions are descriptive and in-sample, computed on the final boosted model's own training rows. Correlated features split credit by the model's internal choices as much as by the data, so directions are more trustworthy than magnitudes.

Figure 4 ranks features, Table 5 lists the top eight, Figure 5 shows the per-strategy spread, and Figure 6 traces attribution against value.

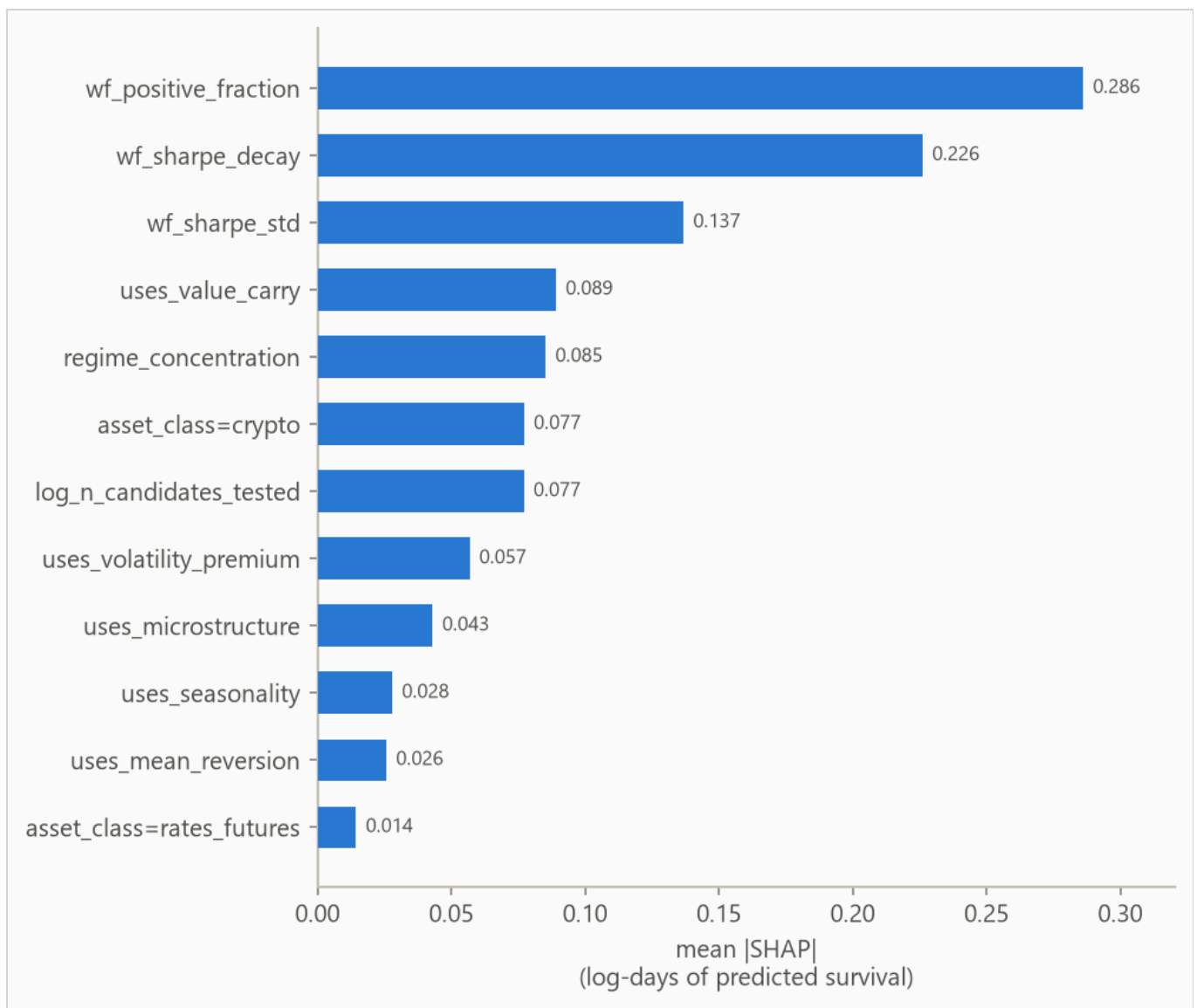


Figure 4. Mean absolute attribution by feature.

Table 5. Top eight features by mean absolute attribution.

FEATURE	MEAN ATTRIBUTION
wf_positive_fraction	0.286
wf_sharpe_decay	0.226
wf_sharpe_std	0.137
uses_value_carry	0.089
regime_concentration	0.085
asset_class=crypto	0.077
log_n_candidates_tested	0.077
uses_volatility_premium	0.057

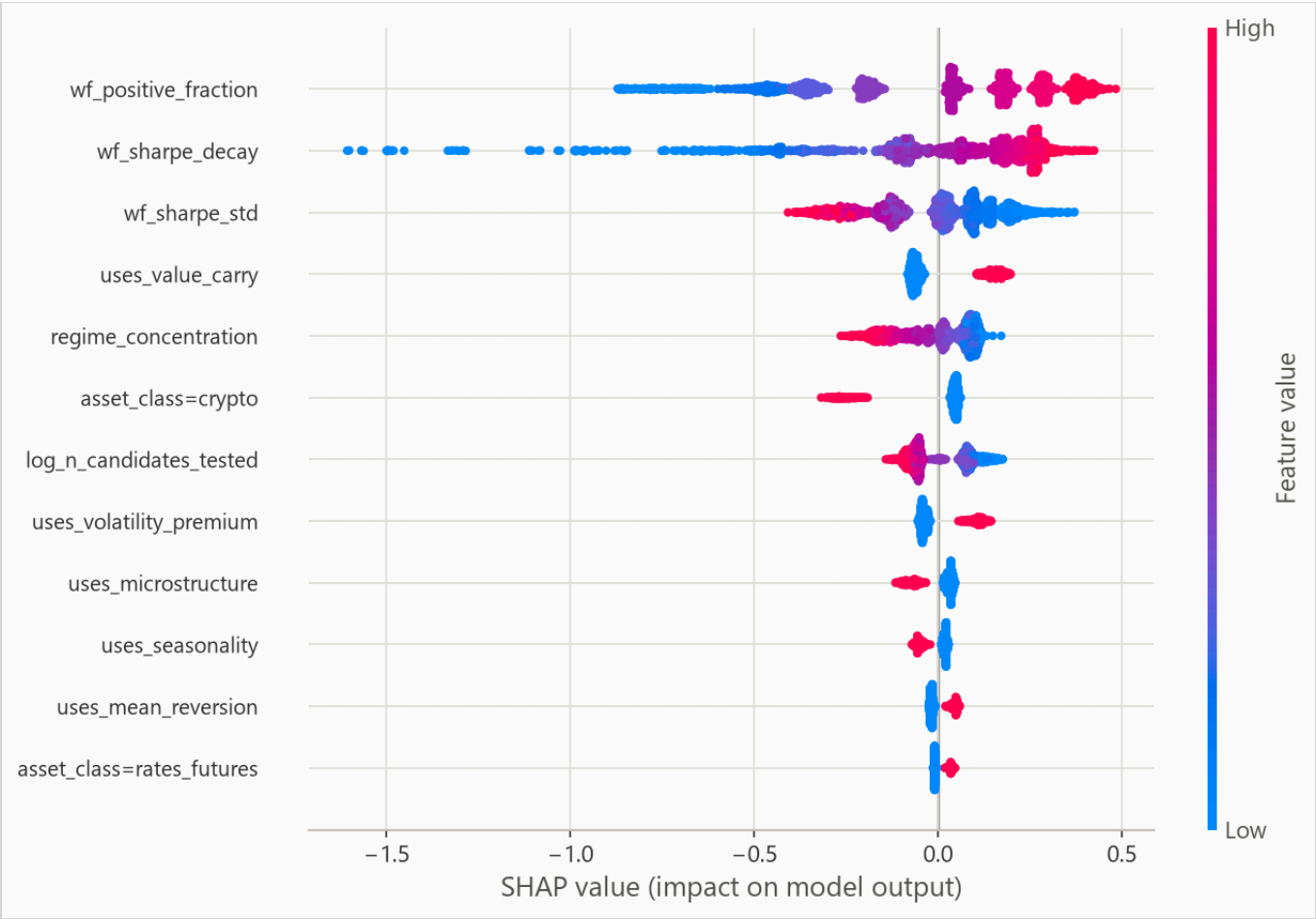


Figure 5. Per-row attributions across the explanation sample. For a yes/no feature, such as a one-hot category flag, the high (red) end simply means the row is in that category.

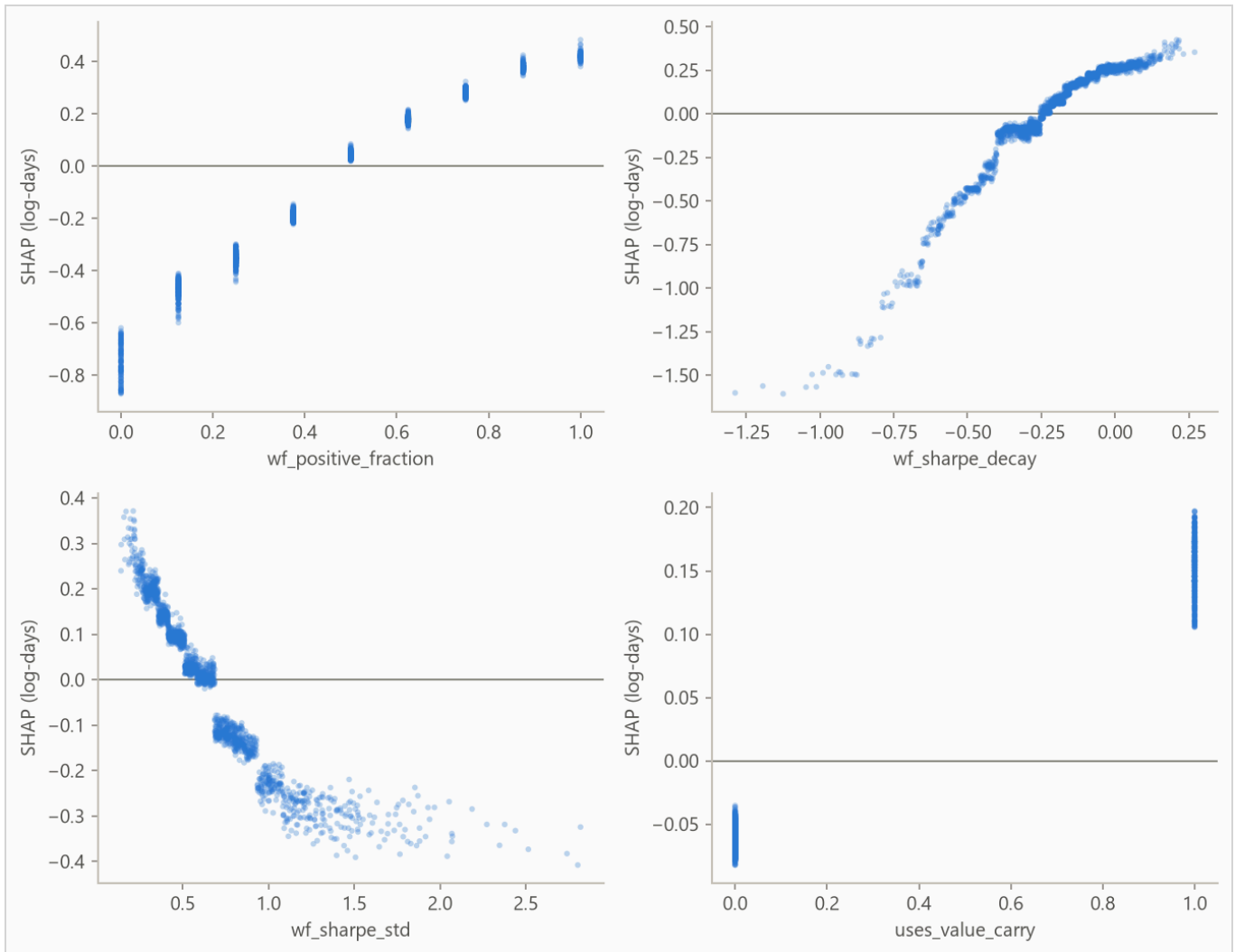


Figure 6. Attribution against feature value for the strongest numeric features. Category flags carry no shape and stay in the ranking above. A run short on numeric features fills the grid with flags instead.

6.2 The Cox baseline

The Cox baseline's drivers are its coefficients, reported as hazard ratios. A hazard ratio is the factor by which one unit of a feature multiplies the hazard, the risk of ending at a given age. The factor is the same at every age, so a ratio above 1 shortens survival, the opposite reading from the attributions above. Figure 7 plots the strongest covariates, ranked by $|z|$, the coefficient over its standard error, and Table 6 lists the top eight.

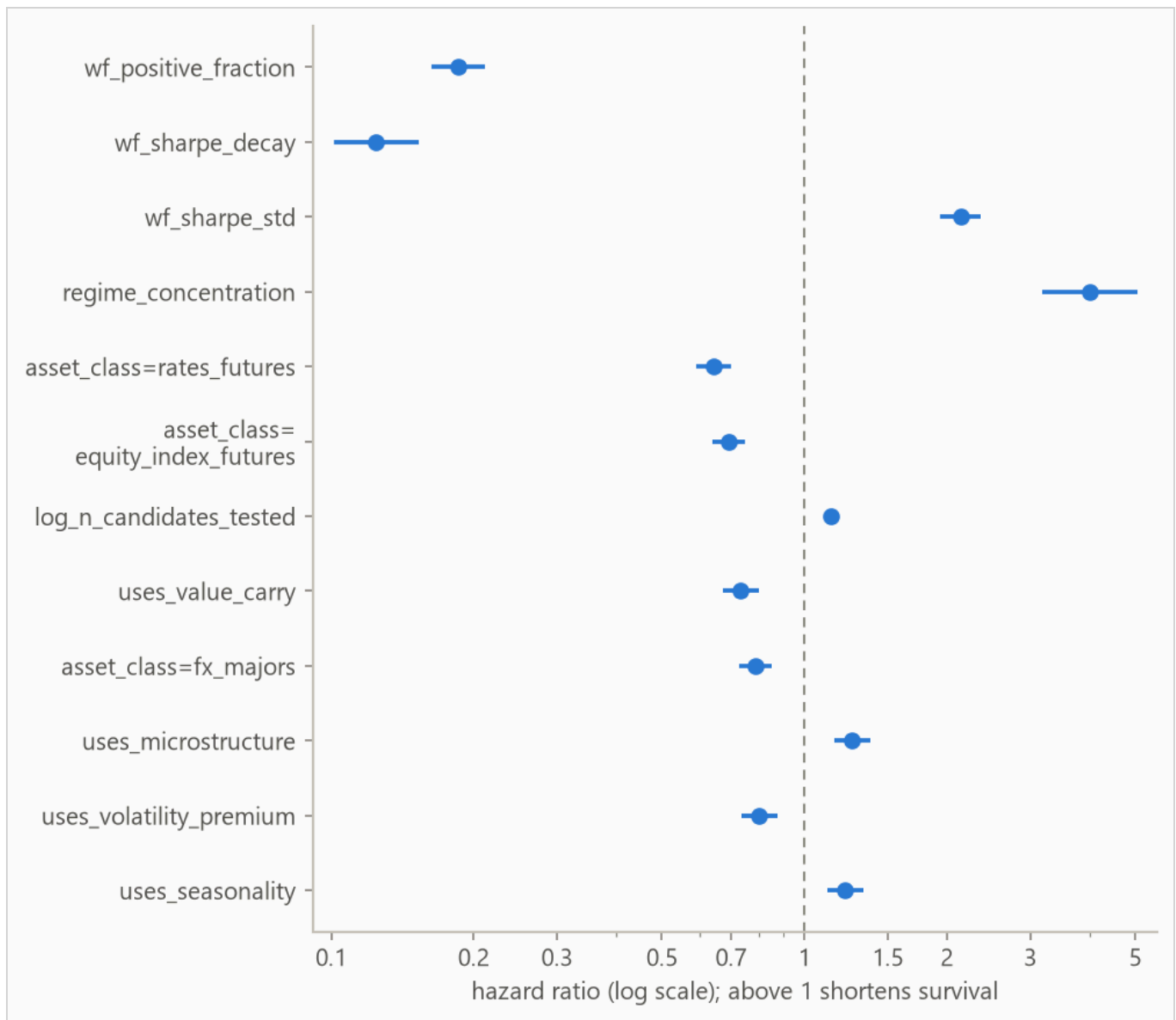


Figure 7. Hazard ratios for the strongest Cox covariates on a log axis, so a doubling and a halving of risk sit the same distance from the dashed no-effect line at 1.0.

Table 6. Top eight Cox covariates by $|z|$. A ratio above 1 raises the hazard and shortens survival. A one-hot flag's ratio compares its category against the column's reference level, the one category dropped from the Cox fit so the remaining flags stay independent. The ridge penalty that stabilizes the fit shrinks coefficients toward zero, so the 95% intervals are approximate.

FEATURE	HAZARD RATIO	95% INTERVAL	Z
wf_positive_fraction	0.185	0.163 to 0.211	-25.7
wf_sharpe_decay	0.125	0.102 to 0.153	-19.9
wf_sharpe_std	2.137	1.934 to 2.360	+14.9
regime_concentration	4.008	3.176 to 5.058	+11.7
asset_class=rates_futures	0.644	0.591 to 0.701	-10.1
asset_class=equity_index_futures	0.691	0.640 to 0.747	-9.4
log_n_candidates_tested	1.138	1.103 to 1.175	+8.0
uses_value_carry	0.734	0.673 to 0.799	-7.1

7. Interpretation

From the run's notes, written by the analyst.

Why ranking by validation Sharpe inverts. The generator installs this inversion deliberately, and the design mimics how real-life strategy decay behaves. Every strategy enters the population by clearing a validation-Sharpe threshold. In the case of our synthetic study that threshold is 0.8. The generator draws an observed Sharpe as the sum of true edge, an overfitting component, and noise, which is the standard winner's-curse setup. Conditioning on that sum exceeding a bar means the survivors with the highest scores are disproportionately the ones whose overfitting and noise broke upward by chance. Past the bar, additional observed Sharpe is more likely inflation than edge. The inflated strategies are the overfit ones, which decay fastest. So higher validation Sharpe actively predicts shorter working life. It scores 0.410 pooled and sits below a coin flip in every fold. The inversion is a property the generator's design implies, and the run confirms the pipeline detects it. The winner's curse is a general statistical effect, so this likely generalizes to real strategy populations. But the data here is purely synthetic, and the run is evidence about the pipeline, not about any real book. This is also what makes the modeling problem worth posing. If the backtest metric ranked survival correctly there would be nothing to add. A meta-model earns its place precisely because selection has already consumed the obvious signal.

Reading the tie. The boosted model and the Cox baseline land at 0.781 and 0.780 on the fold mean, a gap inside the bootstrap interval, which I read as a tie. The synthetic data generator's observable structure is close to additive, which leaves little for trees to find beyond what a penalized linear model in the log-hazard

captures. I report the tie rather than adding interactions to the generator until the headline model wins, because a benchmark tuned until it loses is not a benchmark.

The calibration failure worth keeping. An earlier version of this pipeline selected hyperparameters by concordance and looked correct, then lost to a no-skill forecast on 365-day Brier score. Concordance is invariant to the predictive scale, so the search had settled on a distribution width far too wide and nothing in the training loop objected. That failure is why selection now uses censored log-likelihood and why every report this pipeline produces grades probability quality separately from ranking.

What the attribution shows, checked against the mechanism. The walk-forward statistics that dominate the ranking are not inputs to survival time in the generator. They exist only as noisy proxies for the latent edge-versus-overfit split that is the actual driver, and the model found them and leaned on them. Feature-family and asset-class effects return with the installed signs, among them the crypto shift of -0.30 described in the data note. High walk-forward positive fraction pushes predicted survival up, and a steeply negative walk-forward Sharpe slope pushes it down hard. High fold-to-fold Sharpe dispersion is penalized. All three directions match the generative design, where consistency rises with true edge and falls with overfitting. Validation Sharpe itself is nearly absent from the attribution ranking, which is the winner's curse seen from inside the model. The supportable reading of the magnitudes is that the model uses positive fraction about twice as much as Sharpe dispersion, not that positive fraction is twice as important, since the three walk-forward statistics are correlated proxies of one latent quantity.

Intended use, refit on a real book. Refit on production metadata, the model would be a capital-allocation and review-scheduling prior, never a trade signal. Predicted median lifetime sets the first review date and is one input to initial sizing, and the full curve gives a decay schedule to compare live performance against. It prices day-one information only. Once a strategy is live, realized performance carries information no discovery-time metadata can. Before any such use, the same temporal cross-validation with the same re-censoring has to be repeated on production metadata, and the resulting concordance should be expected to come in lower, with wider intervals.

8. Limitations

The generator encodes a prior, not evidence, so the concordance here is an upper bound on production performance. Lifetimes are drawn and resampled independently, understating real-book uncertainty, and administrative retirement is modeled as independent censoring, with competing risks left as future work. Every number here comes from one generator seed, so data variance and model variance cannot be separated.

Censoring may be informative. The evaluation assumes strategies stop being observed for reasons unrelated to their risk. Early exits of strategies about to end anyway bias survival estimates upward.

The boosted model gives every strategy one curve shape. The predictive scale is a single fitted number, so the model runs a strategy's clock faster or slower but never changes the curve's shape. Per-strategy width is future work.

Harrell's concordance is biased under heavy censoring, usually upward. It scores only the pairs censoring leaves comparable, which under heavy censoring over-represent short observed lifetimes. The most censored test window is 39% censored, so read its rows in Table 2 with the most doubt.

9. Reproducing this run

Python 3.11 or later. From the repository root:

```
pip install -r requirements.txt
python -m pytest
python scripts/run_synthetic_pipeline.py
```

Every number is read from the run's `metrics.json` at build time, and a figure the prose never cites fails the build. `requirements.txt` pins the exact versions the committed numbers came from.

Addendum A. Synthetic ground truth

Effects are installed as fixed shifts on log survival time. Nothing in the generator is proprietary, and the constants are in the run's notes.

The oracle ranking orders strategies by the latent log-time the generator actually used. Nothing is fitted and the ranking predates the noise draw, so its 0.820 bounds every model. The gap to a perfect score is installed noise. A suite test asserts the model never outscores it, since beating perfect information indicates a leak.

Generated from `reports/synthetic/metrics.json` by `scripts/run_build_report.py`. 5,000 strategies drawn at seed 7, 3,000 out-of-time test strategies.